VERILOG

2026

TENBIN:
SECURITY REVIEW REPORT

FEB 7, 2026

Copyright © 2026 by Verilog Solutions. All rights reserved.

Contents

1 Introduction **3**

2 About Verilog Solutions **4**

3 Service Scope **5**
3.1 Service Stages 5
3.2 Methodology 5
3.3 Audit Scope 5

4 Project Summary **6**

5 Findings and Improvement Suggestions **7**
5.1 High 7
5.2 Medium 8
5.3 Low 10
5.4 Informational 17

6 Appendix **18**
6.1 Appendix I: Severity Categories 18
6.2 Appendix II: Status Categories 18

7 Disclaimer **19**

1 | Introduction

Figure 1.1: Tenbin Labs Report Cover

This report presents our engineering engagement with the Tenbin Labs team on their ETHEREUM smart contracts security analysis.

2 | About Verilog Solutions

Founded by a group of cryptography researchers and smart contract engineers in North America, Verilog Solutions elevates the security standards for Web3 ecosystems by being a full-stack Web3 security firm covering smart contract security, consensus security, and operational security for Web3 projects.

Verilog Solutions team works closely with major ecosystems and Web3 projects and applies a quality above quantity approach with a continuous security model. Verilog Solutions onboards the best and most innovative projects and provides the best-in-class advisory services on security needs, including on-chain and off-chain components.

3 | Service Scope

3.1 | Service Stages

Our auditing service includes the following two stages:

- Pre-Audit Consulting Service
- Smart Contract Auditing Service

3.1.1 | Pre-Audit Consulting Service

As a part of the audit service, the Verilog Solutions team worked closely with the Tenbin Labs development team to discuss potential vulnerability and smart contract development best practices in a timely fashion. Verilog Solutions team is very appreciative of establishing an efficient and effective communication channel with the Tenbin Labs team, as new findings were exchanged promptly and fixes were deployed quickly, during the preliminary report stage.

3.1.2 | Smart Contract Auditing Service

The Verilog Solutions team analyzed the entire project using a detailed-oriented approach to capture the fundamental logic and suggested improvements to the existing code. Details can be found under Findings And Improvement Suggestions.

3.2 | Methodology

- **Code Assessment**
  - We evaluate the overall quality of the code and comments as well as the architecture of the repository.
  - We help the project dev team improve the overall quality of the repository by providing suggestions on refactorization to follow the best practices of Web3 software engineering.
- **Code Logic Analysis**
  - We dive into the data structures and algorithms in the repository and provide suggestions to improve the data structures and algorithms for the lower time and space complexities.
  - We analyze the hierarchy among multiple modules and the relations among the source code files in the repository and provide suggestions to improve the code architecture with better readability, reusability, and extensibility.

3.3 | Audit Scope

Our auditing for Tenbin Labs covered the following smart contract code:

- **Controller.sol** – Mint/redeem logic and oracle price tolerance checks.
- **OracleAdapter.sol** – Chainlink oracle price feed adapter.
- **CollateralManager.sol** – Collateral and swap handling, slippage and cap enforcement.
- **StakedAsset.sol** – Staking, cooldown, and vesting logic.
- **DeployTestnet.s.sol** – Testnet deployment scripts.

4 | Project Summary

Tenbin Labs is a Liquid, Yield-Bearing, Multi-Asset Issuance Framework: a unified system for issuing liquid, yield-bearing FX and commodity assets backed by 1:1 collateral pools of USD-stablecoins, without requiring each tokenized asset to maintain its own spot backing.

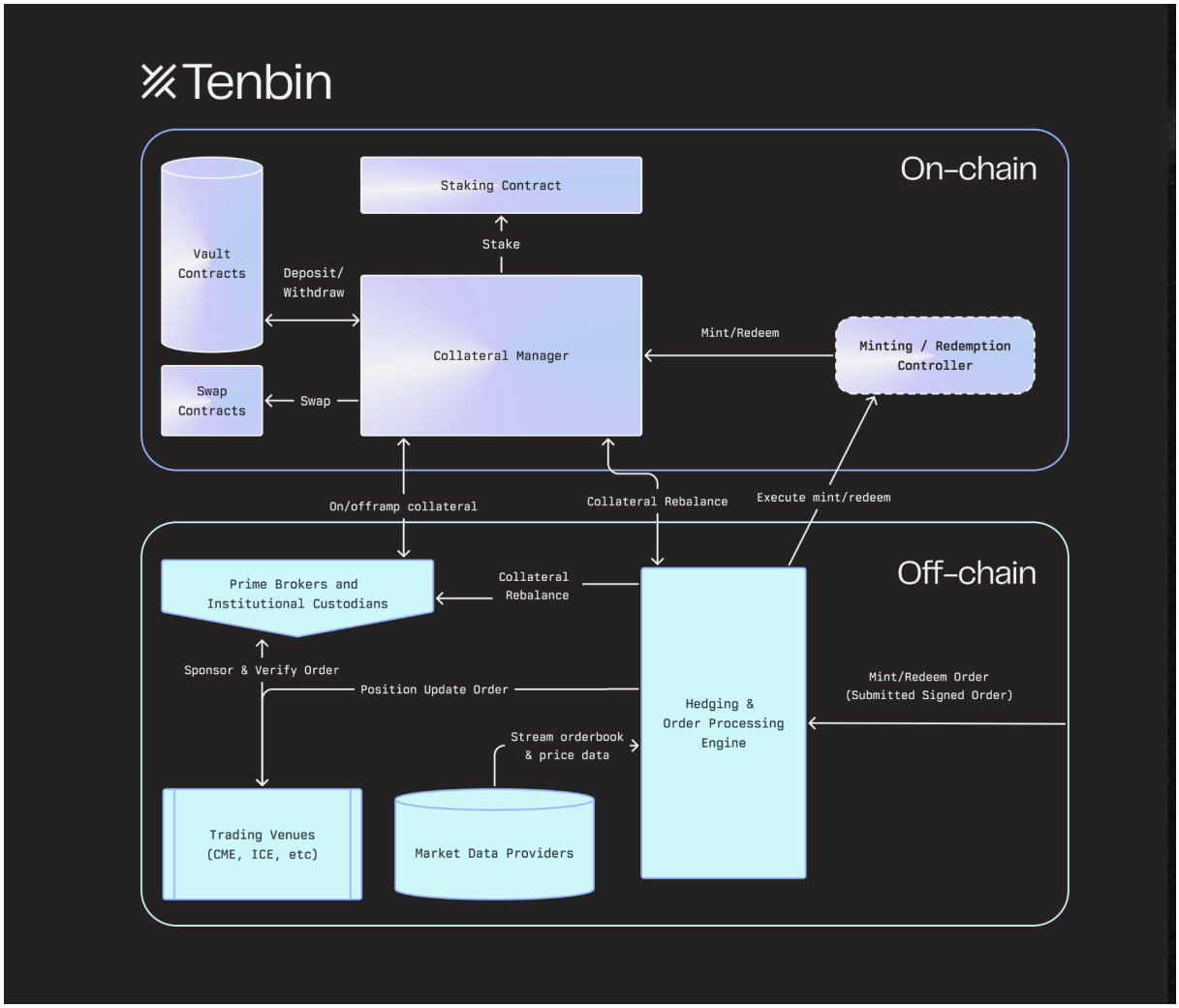

Figure 4.1: Project structure.

5 | Findings and Improvement Suggestions

Severity	Total	Acknowledged	Resolved
High	1	1	1
Medium	2	2	2
Low	7	7	7
Informational	1	1	1

5.1 | High

5.1.1 | Incorrect price as the denominator

Severity	High
Source	Controller.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  The delta should be compared with the price returned by the oracle, not the price of the order. The current implementation uses `price` (order-derived) as the denominator when computing the relative difference, which leads to incorrect oracle tolerance checks.

```
if (oracle.adapter != address(0)) {
    uint256 oraclePrice = IOracleAdapter(oracle.adapter).getPrice();
    uint256 decimals = IERC20Metadata(order.collateral_token).decimals();
    uint256 collateralAmount;
    if (decimals == 18) collateralAmount = order.collateral_amount;
    else collateralAmount = order.collateral_amount * 10 ** (18 - decimals);
    uint256 price = Math.mulDiv(collateralAmount, 1e18, order.asset_amount);
    uint256 difference = price >= oraclePrice ? price - oraclePrice : oraclePrice - price;
    // @audit: should use oraclePrice as denominator
    uint256 delta = Math.mulDiv(difference, 1e18, price);
    if (delta > oracleData.tolerance) revert ExceedsOracleDeltaTolerance();
}
```

- **Recommendation**
  Use the oracle price as the denominator: `uint256 delta = Math.mulDiv(difference, 1e18, oraclePrice);`

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.2 | Medium

5.2.1 | Use of Deprecated `latestAnswer()` in Chainlink Oracle Adapter May Lead to Stale or Invalid Price Validation

Severity	Medium
Source	OracleAdapter.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  The Chainlink `OracleAdapter` retrieves the price via `latestAnswer()`, which has been officially deprecated by Chainlink. This function does not guarantee that the returned price is fresh and does not revert when no valid answer is available (it may return `0`). The returned oracle price is used in the Controller to compute the price delta and enforce oracle tolerance; stale or zero prices can distort the delta and cause incorrect reverts or acceptance.

- **Recommendation**
  Replace `latestAnswer()` with `latestRoundData()`, which provides round information including timestamps. Validate that the timestamp is recent and that the answer is valid before using it in critical validation logic.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.2.2 | Slippage Validation Compares Amounts of Different Assets and Does Not Represent Price Slippage

Severity	Medium
Source	CollateralManager.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  `_verifySlippage` normalizes input and minimum output to 18 decimals but compares amounts of different assets (source token amount vs. destination token amount). Decimal normalization does not equate token values; the computed basis-point slippage is therefore not price slippage but a ratio between unrelated quantities. This can revert valid swaps or fail to enforce intended slippage tolerance.

- **Recommendation**
  Base slippage validation on a meaningful reference, e.g. a router-provided expected output for the swap (`expectedAmountOut`), so that slippage is computed against the same asset and reflects actual price impact.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3 | Low

5.3.1 | Missing the initialization functions

Severity	Low
Source	CollateralManager.sol, StakedAsset.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  `CollateralManager.sol` and `StakedAsset.sol` lack a call to `__AccessControl_init()` in their initializers, which may leave access control in an inconsistent state in upgradeable deployments.
- **Recommendation**
  Call `__AccessControl_init()` in the appropriate `init` functions.
- **Status**
  Resolved and Acknowledged.
- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.2 | Zero Vesting Period Allows Vesting Bypass and Reward Sandwiching

Severity	Low
Source	StakedAsset.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  `setVestingPeriod` allows `newVestingPeriod == 0`. When `vesting.period == 0`, `_pendingRewards` returns `0`, effectively disabling linear vesting and weakening protection against reward sandwiching.
- **Recommendation**
  Disallow a zero vesting period by enforcing `newVestingPeriod >= MIN_VESTING_PERIOD` (remove the `&& newVestingPeriod != 0` exception).
- **Status**
  Resolved and Acknowledged.
- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.3 | Incorrect Decimal Normalization in Price Tolerance Check Breaks Collaterals with Decimals Greater Than 18

Severity	Low
Source	Controller.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  The Controller normalizes collateral to 18 decimals with `order.collateral_amount * 10 ** (18 - decimals)`. When `decimals > 18`, the exponent `18 - decimals` underflows and mint/redeem can revert. Although `setIsCollateral` may restrict decimals to 6–18, the normalization should support both `decimals < 18` and `decimals > 18`.

- **Recommendation**
  Use the same pattern as in `CollateralManager._normalizeTo18`: if `decimals == 18` return `amount`; if `decimals < 18` return `amount * 10 ** (18 - decimals)`; otherwise return `amount / 10 ** (decimals - 18)`.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.4 | Missing Destination Cap Check on Actual Swap Output May Cause Cap Underflow

Severity	Low
Source	CollateralManager.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  The swap path checks `params.minReturnAmount` against `dstTokenCap` but does not validate the actual received amount. If the swap returns more than `dstTokenCap`, the update `dstTokenCap - (balanceAfter - balanceBefore)` can underflow and revert.

- **Recommendation**
  After the swap, set `received = balanceAfter - balanceBefore`, require `received >= params.minReturnAmount` and require `received <= dstTokenCap` before decrementing the destination cap.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.5 | Missing Upper-Bound Validation in `setSwapTolerance` Allows Invalid Tolerance Values

Severity	Low
Source	CollateralManager.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  `setSwapTolerance` does not enforce an upper bound on `tolerance`. With basis points using `BASIS_PRECISION = 10_000`, a tolerance above 10 000 is invalid and can effectively disable slippage protection.
- **Recommendation**
  Require `tolerance <= BASIS_PRECISION` (and revert with a dedicated error, e.g. `ToleranceExceedsMax()`) before storing the new tolerance.
- **Status**
  Resolved and Acknowledged.
- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.6 | Inconsistent Rescue Functions Between Controller and CollateralManager

Severity	Low
Source	CollateralManager.sol; Controller.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  The Controller has `rescueToken` for ERC20 recovery; the CollateralManager only has `rescueEther`. ERC20 tokens sent to CollateralManager cannot be recovered, creating asymmetric recovery and risk of locked funds.
- **Recommendation**
  Implement both ERC20 and ETH rescue in both contracts so recovery capabilities are consistent.
- **Status**
  Resolved and Acknowledged.
- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.3.7 | Donation attack is still possible

Severity	Low
Source	DeployTestnet.s.sol, StakedAsset.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  `StakedAsset` documents that a minimum stake should be made in the same transaction as deployment to avoid a first-depositor donation attack. The testnet deployment script does not perform this initialization mint. Impact is low due to OpenZeppelin's virtual asset protections.

- **Recommendation**
  Perform the minimum initial stake in the same transaction as contract deployment, or document and accept the residual risk.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

5.4 | Informational

5.4.1 | Incorrect Comment Suggests Calling `withdraw()` After Cooldown Instead of `unstake()`

Severity	Informational
Source	StakedAsset.sol
Commit	db853f978b3481b64453661537b7d39179a78d70;
Status	Resolved;

- **Description**
  A comment states that after cooldown users can call `withdraw()` to claim their assets. In reality, users must call `unstake()`, since assets are moved to the silo during cooldown and `withdraw()` is disabled while cooldown is active. The comment can mislead integrators and users.

- **Recommendation**
  Update the comment to instruct users to call `unstake()` after the cooldown is completed.

- **Status**
  Resolved and Acknowledged.

- **Fixed in**
  `db853f978b3481b64453661537b7d39179a78d70`.

6 | Appendix

6.1 | Appendix I: Severity Categories

Severity	Description
High	Issues that are highly exploitable security vulnerabilities. It may cause direct loss of funds / permanent freezing of funds. All high severity issues should be resolved.
Medium	Issues that are only exploitable under some conditions or with some privileged access to the system. Users' yields/rewards/information is at risk. All medium severity issues should be resolved unless there is a clear reason not to.
Low	Issues that are low risk. Not fixing those issues will not result in the failure of the system. A fix on low severity issues is recommended but subject to the clients' decisions.
Information	Issues that pose no risk to the system and are related to the security best practices. Not fixing those issues will not result in the failure of the system. A fix on informational issues or adoption of those security best practices-related suggestions is recommended but subject to clients' decision.

6.2 | Appendix II: Status Categories

Severity	Description
Unresolved	The issue is not acknowledged and not resolved.
Partially Resolved	The issue has been partially resolved
Acknowledged	The Finding / Suggestion is acknowledged but not fixed / not implemented.
Resolved	The issue has been sufficiently resolved

7 | Disclaimer

Verilog Solutions receives compensation from one or more clients for performing the smart contract and auditing analysis contained in these reports. The report created is solely for Clients and published with their consent. As such, the scope of our audit is limited to a review of code, and only the code we note as being within the scope of our audit is detailed in this report. It is important to note that the Solidity code itself presents unique and unquantifiable risks since the Solidity language itself remains under current development and is subject to unknown risks and flaws. Our sole goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies. Thus, Verilog Solutions in no way claims any guarantee of security or functionality of the technology we agree to analyze.

In addition, Verilog Solutions reports do not provide any indication of the technology's proprietors, business, business model, or legal compliance. As such, reports do not provide investment advice and should not be used to make decisions about investment or involvement with any particular project. Verilog Solutions has the right to distribute the Report through other means, including via Verilog Solutions publications and other distributions. Verilog Solutions makes the reports available to parties other than the Clients (i.e., "third parties") – on its website in hopes that it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.